The Swarm

An LLM captain commanding many agents under fog of war

October 30, 2026

Abstract

The scout/fighter fleet proved delegation and amalgamation under partial observability, but its captain's choices were *forced*: one scout to the nearest frontier, one fighter to the one blocking monster, march when the exit is reachable. Nothing was decided, so making that captain a model would have been theatre. The *swarm* removes the forcing. Five scouts, two fighters, and a captain explore a large branching maze in which the captain faces genuine *allocation under partial information*: which scout to send where, which fighter to which newly discovered monster, and when to stop exploring and commit. The captain is an LLM. It sits at the *orchestration* layer, not the leaf, commanding a real delegation hierarchy in which it *cannot* cheat, because every order it gives is decoded against the fog it has been told and bounded by the same reason-gate as a leaf. The construction sharpens three claims. The amalgamation becomes a true *N-ary* join, and the captain's map is well-defined precisely because that join is commutative, associative and idempotent. The scheduler is a tick loop whose determinism rests on a fixed agent order and serial execution. And the model-as-oracle-behind-a-decoder argument, which previously guarded a single leaf, now guards the commander: a hallucinated order is a refused position, never an escape hatch. Everything scripted is deterministic — `verify.sh` replays it to the same bytes, website included — and the whole thing has been run live against a free model.

1 The one idea

A container is a pair (P, R): a set of prompts P and, for each p, a type $R\,p$ of permitted responses. A program is a dependent function $(p : P) \to R\,p$. An *open agent* is a morphism of containers: a map $u : P \to Q$ sending a prompt down to a sub-prompt (control flows *down*) and a map $f : (p : P) \to T(u\,p) \to R\,p$ sending the sub-answer back up (data flows *up*, contravariantly). A captain delegating to officers, an interpreter calling a device driver, an LLM calling a tool: one construction.

The fleet put a *scout* at one leaf of this hierarchy and asked whether a model could make the scout's junction choices. It could. The swarm asks the sharper question: can a model sit at the *root* — the commander that delegates to the whole hierarchy — and still be bound by the type discipline rather than escaping it? The answer is the subject of this note. The load-bearing fact is that a model is an untyped String → String; presenting it as a typed planner plan : $S \to P\,s$ is a *promise*, and the promise is kept by a validator. A reply that does not decode throws; it never defaults into a checked-looking value.

2 What the swarm adds over the fleet

The swarm reuses the fleet's world, fog, pathing, two-layer log and combinators wholesale. Four things are new.

Contention	Five scouts over one fog: the captain must fan them to *different* regions. Two fighters over several monsters: it must allocate, and choose which monsters are worth a fighter at all.
N-ary join	The captain folds *all five* scout reports into its fog each step. The fleet merged one at a time and never exercised associativity; the swarm folds over N, and the fold is well-defined only because the join's laws hold.
Tick scheduler	The fleet's two-phase round (one scout to completion, then one fighter, then march) does not survive contention. A pool of eight agents is advanced one tick at a time, and the captain re-plans around whoever has gone idle.
Model at the root	The captain's decision is a seam. The default is a deterministic greedy planner; `--live` swaps in a model. The decoder and reason-gate that guarded the scout now guard the commander.

3 The N-ary amalgamation

The captain's entire state is a `FogMap`: a flat semilattice of terrain facts (*unknown* $\sqsubseteq$ $\{$*floor*, *wall*, *exit*$\}$, with a top *conflict* so the join is total) lifted cellwise, plus a revisable dictionary of monster positions. The join `mergeFog` is idempotent, commutative, associative and monotone — law-tested in `fog_test.ts`. This is the treasury/org-chart "real algebra" pole, not a tuple gather.

The fleet merged one scout report at a time, so it only ever used the join as a binary operation and never relied on associativity. The swarm gathers five reports in a single planning step and folds them all in:

$$\text{fog}' \;=\; \text{fog} \;\sqcup\; \bigsqcup_{i=1}^{5} \text{fogOf}(r_i),$$

implemented as `reports.reduce((f, r) => mergeFog(f, fogOf(r)), base)`. Because $\sqcup$ is commutative and associative, the result does not depend on the order the reports arrive, and because it is idempotent, re-merging a report is harmless. This is not a nicety; it is what *licenses* the scheduler to process the scouts in a fixed serial order (next section) without the captain's map depending on that order. The order-independence is proven end-to-end on sightings taken off the real maze in `amalgamate_test.ts`: folding six reports in natural, reversed and shuffled orders yields the identical fog.

A subtlety worth stating: within one tick each scout navigates on the fog *snapshotted at the start of the tick* plus its own new sightings — it does *not* see the other scouts' sightings from the same tick. So the five reports are genuinely independent (a tensor, $\otimes$, not a product), and the join combines them afterward. Kills are applied outside the fold (a monster the fighter clears is retracted from the dictionary), keeping the fold itself a pure monotone climb.

4 The tick scheduler and the determinism contract

Each agent carries a runtime state — `idle`, `exploring`, `travelling`, `fighting`, `done` — and, for scouts and fighters, a standing order. One tick advances every active agent by its speed, folds the scouts' reports into the fog, applies any kills, and — when there is something to decide — asks the captain to re-plan.

The rule the whole thing rests on

Agents are processed in a **fixed id order** within a tick, and their gated CLI calls run **serially**. `tensorC` means "all five scout reports are owed," not "executed concurrently." A `Promise.all` over agents that mutate the shared world file would make the move order — and therefore the audit — non-reproducible. The join's order-independence says the *fog* would be the same; the *log* would not. So we serialise.

One planning step reads as $\mathrm{seqC}(\mathrm{tensorC}(\mathrm{scout}_1 \ldots \mathrm{scout}_5), \mathrm{dispatch})$: gather all five reports (the serial tensor), fold them into the fog (the join), then let the captain choose the fighter dispatches and the march (the dependent `next`). The captain commands *only idle agents* and never reassigns one mid-order, so re-planning can happen as often as we like without thrashing a standing order.

Event-gated planning. Re-planning every tick is cheap for the scripted captain but would mean a model call per tick on `--live`. The scheduler therefore consults the captain only when there is genuinely something to decide. The gate oracle is the *greedy* plan itself: if the keyless, deterministic scripted captain finds nothing to assign — no idle scout with an open frontier, no idle fighter with a reachable monster, no safe march — then an idle fighter held in reserve is not an event, and the captain is not consulted. This bounds the LLM to a couple of dozen calls over a solve rather than a couple of hundred, and it is *byte-identical* for the scripted captain: on a skipped tick its plan was empty and changed no move, no log and no frame.

5 The scripted captain

The default `CaptainBrain` is a greedy planner, deterministic and keyless, so `verify.sh` stays byte-identical. It operates in two modes.

Explore. Fan idle scouts across *open* frontiers — those reachable without a fight, assigned nearest-first so the scouts spread. While open frontier remains, spend no fighter: exploring is free. Only when exploration *stalls* (every reachable frontier sits behind a monster) dispatch the nearest idle fighter to the nearest reachable monster, to reopen the map.

Commit. Once a known-floor corridor to the exit exists, stop exploring and clear only the monsters *on that corridor*; ignore every other monster. March the instant a monster-free route is known.

The honest limitation is worth stating plainly. From the fog alone, a monster that gates the half holding the exit and a monster that gates the decoy half are *indistinguishable*: both have unknown cells behind them, because the monster is exactly *why* that ground is dark. So the greedy captain cannot pre-judge which gate to skip before clearing one — it must open both, which is precisely what puts *both fighters* to work at once. What it *can* do honestly is commit-mode prioritisation: once a monster-free route to the exit is known, it leaves every off-route monster alone — the decoy half and the monsters deep in it. Sharper judgement — inferring which gate is the decoy before clearing it, and holding a fighter back — is where a model has room to do better, and room to do worse.

This is why the shipped maze forks at a central start into two large halves rather than running a single corridor. An earlier draft placed the monsters on one linear path to the exit; in a perfect maze the route to the second passes through the first, so the monsters surfaced strictly one at a time, a single fighter cleared them all, and the second fighter never moved — the allocation the swarm is built to show never arose. The fork forces it: two gates at equal shallow depth, discovered together, neither distinguishable from fog, so the captain allocates both fighters in one step.

6 The LLM captain

`modelCaptainBrain` is a second `CaptainBrain` behind the same seam; the scheduler cannot tell it from the greedy default. It reuses the fleet's model stack — the neutral `ModelClient` boundary and the free, OpenAI-compatible Groq backend with its 429 backoff — verbatim.

Each step the captain's fog and roster are serialised into a prompt (an ASCII map with the agents, monsters and frontier marked, plus explicit lists of idle agents, known monsters and some frontier coordinates), and the model is offered four tools: `assign_scout`, `dispatch_fighter`, `march`, `hold`. The set is the permission grant; there is no "do anything." Every item of the reply is then *decoded against the real state*:

- a scout id that is not an idle scout — dropped;
- a target cell that is not reachable known floor — dropped;
- a fighter id that is not an idle fighter, or a monster id not known-alive — dropped;
- a `march` when no monster-free route is known — dropped;
- any order with an empty reason — dropped (the reason-gate, on orders).

An order that does not decode is a refused position, never an escape hatch: the model can mis-prioritise, but it cannot name a scout that does not exist, send a unit into the dark, or march through a wall.

Anti-stall, not anti-idle. The fallback is the delicate part. If the greedy plan always filled every slot the model left empty, the model could never *choose* to leave a fighter in reserve — its prioritisation would be silently overridden and it would behave exactly like greedy. So the rule is: honour the model's valid orders as-is, including the choice to leave agents idle; fall back to greedy *only when the model's whole plan is unproductive while real work remains.* A mis-prioritising model is slower but honoured; a lazy or stuck model can never deadlock the swarm. This is the orchestration-layer analogue of the scout's "off-menu reply → BFS fallback," raised one level and with the stakes raised to match.

7 The two-layer log

Nothing about the audit changes, which is the point. Every move and attack by all five scouts, both fighters and the captain passes the registry allowlist and the reason-gate, writes a committed effect to Layer 1 (the ship's log, *what*) and a write-ahead intent record to Layer 2 (the intent-log, *why*), joined by a correlation id. Reconciliation cross-checks the two: a committed mutation with no logged reason is an alarm that must never occur, and a refused mutation is visible only in Layer 2 — information the committed log alone could never show. On `--live` the id and clock seams are the *real* ones, never the fixed stand-ins: a live audit must carry true timestamps, or it is a false record of the very thing reconciliation exists to check.

8 Live validation

Run against Groq's `openai/gpt-oss-120b`, the LLM captain fanned the five scouts to different frontiers — its own reasons, e.g. *"monster blocks potential route to exit"* — and then, in a single planning step, dispatched `fighter-1` to one gate and `fighter-2` to the other: **both fighters allocated at once**, the decision the fork exists to force. It cleared the two gates and the one monster on the exit route, left the decoy monsters alone, and reached the exit in **136 ticks across 18 planning calls**, with **493 of 1189 cells never revealed** to it and reconciliation

clean (**0 alarms**) over a real audit trail. The model commanded as well as the greedy captain here; the claim the construction actually makes is stronger and humbler at once: it *could not* command worse than refused.

9 What is, and is not, decided

The swarm's thesis is that a language model can run a real delegation hierarchy in which the type discipline, not trust, is what bounds it — and that the bound holds whether the model is strong, weak or adversarial, because a bad order is a refused position. That is demonstrated. What is *not* claimed is that the model orchestrates *better* than the greedy planner; on this maze the two land in the same place, and "the model decides better" is a separate, harder, model-dependent question the guardrails deliberately do not need answered. The honest shape of the result is the same as the scout's: a free model can slow a solve or mis-prioritise within it, but it never breaks the solve and never escapes the audit.